# Daily Quant Digest: 2026-10-19

**Today's topics**

| Track | Topic | Difficulty |
|---|---|---|
| Probability & Statistics | Restart, day 1: conditional probability and Bayes, made intuitive (the "shrinking room" and the "evidence multiplier") | Medium |
| Derivatives | The Greeks explained like driving a car, plus how theta and gamma trade off, on a real EURUSD trade | Medium |
| Algorithms & DS | Intervals and sweep line: the skyline problem | Hard |
| Music | Pink Floyd, "Wish You Were Here" (1975) | |

## 1. Probability & Statistics: Conditional Probability and Bayes, Intuitively

*We are restarting the probability track from the beginning, with a new style: one big idea per day, explained with pictures in your head first, and formulas second.*

### 1.1 The one idea: conditioning means shrinking the room

Imagine every possible outcome is a person standing in a big room. A probability is just "what fraction of the room is wearing a red hat".

Now someone tells you a fact: "the outcome is in the left half of the room". You do not need any new rule. You simply **send everyone else out** and ask the same question again: what fraction of the *people still in the room* wear red hats?

That is all conditional probability is:

$$P(A \mid B) = \frac{P(A \text{ and } B)}{P(B)} = \frac{\text{people who are both } A \text{ and } B}{\text{people who are } B}.$$

The denominator $P(B)$ is the size of the new, smaller room. Dividing by it just re-scales so the new room adds up to 100%.

**Why this matters in interviews.** Almost every "trick" question is a question about *which room you are really in*. Most mistakes come from keeping people in the room who should have been sent out, or sending out people who should have stayed.

### 1.2 Bayes' rule: flipping the question around

Often you know the easy direction ("if a trader is cheating, how likely is an alert?") but you want the hard direction ("given an alert, how likely is it that the trader is cheating?"). These are **different rooms**, so they give different answers.

**The cleanest way to do Bayes in your head: count people, not probabilities.**

*Example (compliance surveillance at a bank).* 1 trader in 1,000 is cheating. The system catches 99% of cheaters, but also wrongly flags 2% of honest traders. A trader is flagged. How likely is it that they cheated?

Picture 1,000 traders:

- 1 cheater, who gets flagged (0.99, call it 1 person).
- 999 honest traders, of whom 2% are flagged: about **20 people**.

So about 21 people are flagged and only 1 of them is a cheater. The answer is roughly $1/21 \approx 4.7\%$ (exactly $0.99/(0.99 + 19.98) = 4.72\%$).

The "99% accurate" alarm is wrong about 95% of the time. Nothing is broken: the room of flagged people is dominated by the huge crowd of honest traders, because there are so many of them. This is called **base rate neglect**, and it is the single most common mistake in probability (and in reading backtests: a strategy that "looks great" is far more likely to be one of the many lucky duds than one of the few real edges).

### 1.3 The odds form: evidence is a multiplier

Bayes becomes almost mental arithmetic with **odds** instead of probabilities.

$$\underbrace{\frac{P(H \mid E)}{P(\text{not } H \mid E)}}_{\text{odds after}} = \underbrace{\frac{P(H)}{P(\text{not } H)}}_{\text{odds before}} \times \underbrace{\frac{P(E \mid H)}{P(E \mid \text{not } H)}}_{\text{likelihood ratio}}$$

**Analogy: a court case.** You walk in with a starting belief (the odds before). Each piece of evidence is a witness who says "this is $L$ times more likely if the person is guilty than if they are innocent". You multiply your odds by $L$. Independent witnesses multiply one after another.

Surveillance example: odds before = $1 : 999$. Likelihood ratio = $0.99/0.02 = 49.5$. Odds after = $49.5 : 999 \approx 1 : 20$. Same answer as counting people, but you never wrote a fraction tree.

Two things to remember:

- A likelihood ratio above 1 pushes you towards $H$; below 1 pushes you away; exactly 1 means the evidence is useless, however "accurate" it sounds.
- Strong evidence cannot rescue a tiny prior in one step. You need *repeated* independent evidence: that is why compliance teams look for patterns, not single alerts.

### 1.4 Interview question

> A bag has 10 coins. 9 are fair, 1 has heads on both sides. You pull out a coin at random, flip it 3 times, and get 3 heads.
> (a) What is the probability you are holding the double-headed coin?
> (b) What is the probability the next flip is heads?
> (c) How many heads in a row do you need before you are at least 90% sure it is the double-headed coin?

### 1.5 Solution

**(a) Count people.** Imagine doing the experiment 80 times (80 is chosen so the numbers come out whole).

- About 8 times you pick the double-headed coin. It shows HHH every time: **8**.
- About 72 times you pick a fair coin. HHH happens 1 time in 8: **9**.

Only the HHH cases are left in the room: $8 + 9 = 17$. The double-headed coin is 8 of them.

$$P(\text{double} \mid HHH) = \frac{8}{17} \approx 47\%.$$

**Same thing with odds.** Odds before: $1 : 9$. Each head is twice as likely with the double coin (1 vs 1/2), so 3 heads give a likelihood ratio of $2^3 = 8$. Odds after: $8 : 9$, so probability $8/17$.

**(b) Next flip.** Split the room you are now in:

$$P(H) = \frac{8}{17} \times 1 + \frac{9}{17} \times \frac{1}{2} = \frac{16 + 9}{34} = \frac{25}{34} \approx 73.5\%.$$

Common mistake: answering $1/2$ ("coins have no memory"). The coin has no memory, but **you** have learned something about which coin it is.

**(c)** After $k$ heads the odds are $2^k : 9$. We need odds of at least $9 : 1$, so $2^k \geq 81$, so $k = 7$ (since $2^6 = 64$ is not enough). Check: $k = 6$ gives $64/73 = 87.7\%$; $k = 7$ gives $128/137 = 93.4\%$.

**Sanity checks.** The answer to (a) sits between the prior (10%) and 100%, and it moved a lot because the evidence was strong (8 to 1). The answer to (b) sits between 50% (definitely fair) and 100% (definitely double). Always check that an answer lands between its "extreme cases".

## 2. Derivatives: The Greeks, Explained Like Driving a Car

### 2.1 The setup

You are on the FX options desk. You **buy** a 1-month EUR call (the right to buy EUR at 1.1000), on EUR 10 million. Spot EURUSD is 1.1000, implied vol is 8%. You pay about **USD 110,000** of premium.

Why would anyone buy this? A European company that has to pay USD bills, or a fund that thinks EUR will rally, wants upside without unlimited downside. The bank sells them the option and then has to manage the risk. The Greeks are the dashboard for doing that.

### 2.2 The dashboard

Think of the option as a car, and the P&L as the distance you travel when spot moves.

| Greek | Car analogy | What it means here | Our trade |
|---|---|---|---|
| **Delta** | Speed | "How much EUR do I *effectively* own right now?" | 53%, so it behaves like owning EUR 5.3M |
| **Gamma** | Acceleration | How fast delta changes when spot moves | Delta goes up by about EUR 1.7M for each 1% rally |
| **Theta** | Fuel you burn each day | What you lose each day if nothing happens | about USD 2,000 per day |
| **Vega** | Sensitivity to the weather | What you gain if the market gets more nervous (vol up 1 point) | about USD 12,500 per vol point |

**Delta: the option is a partial position.** An at-the-money call is "half in, half out": if EUR rallies a lot you will own the full 10M, if it falls you will own nothing. Right now it behaves like 53% of the position. To have no view on direction, the desk **sells EUR 5.3M spot**. This is called delta hedging.

**Gamma: the position resizes itself in your favour.** If EUR rallies, the option becomes more like owning EUR, so your delta rises: you automatically own *more* of the thing that is going up. If EUR falls, your delta shrinks: you automatically own *less* of the thing going down. It is like a shop that stocks more umbrellas the moment it starts raining, and fewer when the sun comes out. That is why being long an option feels "convex": you win more on the way up than you lose on the way down.

**Theta: the rent.** That nice behaviour is not free. Every day you pay rent, about USD 2,000. Like an ice cube, the option melts faster near the end: with 1 day left, the same option would burn about USD 9,500 a day.

**Vega: insurance before a typhoon.** In Hong Kong, when a typhoon is forecast, the price of insurance goes up *before* any damage happens. Options are insurance on market moves. When markets get nervous (an election, a central bank meeting), implied vol rises and every option you own is worth more, even if spot has not moved yet.

### 2.3 The key trade-off: you pay rent to own big moves

After you delta hedge, direction no longer matters. What is left is a simple bet:

> **You pay theta every day. Gamma pays you back whenever spot moves a lot, in either direction.**

How big a move do you need to break even? The market is charging rent as if it expects EURUSD to move about

$$\sigma \times \sqrt{\tfrac{1}{365}} = 8\% \times 0.052 = 0.42\% \text{ per day} \approx 46 \text{ pips}.$$

- If EURUSD moves **more than about 46 pips** a day on average, the gamma gains beat the rent. You win.
- If it moves **less**, you lose the rent slowly.

So buying an option and delta hedging is really a bet that **actual (realized) volatility will be higher than the implied volatility you paid**. Selling an option is the opposite: you collect rent, like an insurance company, and you lose when the typhoon actually hits.

### 2.4 Two days in the life of the trade (delta hedged)

**A quiet day.** EURUSD does not move, vol does not move. You lose about **USD 2,000**: the rent. Five quiet days cost about USD 10,000.

**A big day.** Overnight, EURUSD rallies 1% to 1.1110 (that is 110 pips, more than twice the breakeven) and vol rises to 8.5% because the market got nervous.

| Piece | Why | USD |
|---|---|---|
| Gamma | spot moved a lot, and your position resized itself in your favour | +9,518 |
| Vega | the market got more nervous, so insurance is worth more | +6,259 |
| Theta | one day of rent | −1,978 |
| Other small effects | | −982 |
| **Total** | | **+12,817** |

(The direction piece, about +58,500 on the option, was cancelled by the EUR 5.3M you sold as a hedge. That is the point of hedging: you are no longer betting on direction.)

Notice the gamma gain: $\frac{1}{2} \times \text{gamma} \times (\text{move})^2$. The gain grows with the **square** of the move. A move twice as big earns four times as much. That is why one wild day can pay for weeks of rent, and why sellers of options fear wild days.

### 2.5 What goes wrong in the real world

- **Selling insurance, then the typhoon hits.** On 15 January 2015 the Swiss National Bank suddenly removed its EURCHF floor and the franc jumped about 30% in minutes. Anyone short options (short gamma) could not hedge in time. Years of collected rent disappeared in one morning.
- **Expiry day "pinning".** With hours to go, an at-the-money option's delta can swing from 0% to 100% on a few pips. The car is now a go-kart with no brakes: tiny moves force huge hedges.
- **Vol is not one number.** 1-week vol and 1-year vol can move very differently, so desks track vega by expiry, not as one total.

### 2.6 Likely interview questions

1. **"You own a delta-hedged option and the market does not move all week. What happens?"** You lose theta every day. Owning options is paying rent for the chance of big moves; no moves means you just paid rent.
2. **"Is delta the probability that the option ends in the money?"** Roughly, for short-dated options, but not exactly. The risk-neutral probability of finishing in the money is $N(d_2)$, while delta uses $N(d_1)$. They differ more for long-dated or high-vol options.
3. **"Why is gamma highest for at-the-money options close to expiry?"** Delta must jump from 0 to 100% somewhere near the strike. Close to expiry there is no time left to make that change gradually, so the jump becomes very steep, and gamma is the steepness.

---

## 3. Algorithms & Data Structures: The Skyline Problem (Sweep Line)

### 3.1 Problem

You are given $n$ buildings, each a rectangle $[\text{left}_i, \text{right}_i) \times [0, \text{height}_i]$ on flat ground. Return the **skyline**: the outer contour of their union, as a list of "key points" $[x, y]$ sorted by $x$. Each key point is the left end of a horizontal segment of the contour; the last key point always has $y = 0$ and marks where the rightmost building ends. There must be **no consecutive key points with the same height** (merge them).

Constraints: $1 \le n \le 10^4$, $0 \le \text{left}_i < \text{right}_i \le 2^{31} - 1$, $1 \le \text{height}_i \le 2^{31} - 1$, buildings sorted by left edge (we will not rely on that).

Example 1:

```
Input:  [[2,9,10],[3,7,15],[5,12,12],[15,20,10],[19,24,8]]
Output: [[2,10],[3,15],[7,12],[12,0],[15,10],[20,8],[24,0]]
```

Example 2 (touching buildings with the same height merge):

```
Input:  [[0,2,3],[2,5,3]]
Output: [[0,3],[5,0]]
```

### 3.2 From first principles

**Brute force.** The skyline height at $x$ is $h(x) = \max\{\text{height}_i : \text{left}_i \le x < \text{right}_i\}$ (0 if none). The function $h$ can only change at a building edge. Collect all $2n$ edge coordinates, sort and deduplicate them, and for each one compute $h(x)$ by scanning all buildings; emit $(x, h(x))$ whenever it differs from the previous height. That is $O(n^2)$: about $2 \times 10^8$ operations at $n = 10^4$, borderline. (A pixel grid over $x$ is hopeless: coordinates go to $2^{31}$.)

**Insight 1: sweep line.** Move a vertical line from left to right. At each edge, the set of "active" buildings (those the line currently cuts) changes by one insertion (left edge) or one deletion (right edge). Between edges, nothing changes. So the problem reduces to maintaining a dynamic set of heights under insert and delete, with **max queries**. This is the general sweep-line recipe: convert intervals into $2n$ sorted events and maintain a data structure that summarizes the active intervals.

**Insight 2: the right data structure.** We need insert, delete an arbitrary element, and find max, all in $O(\log n)$:

- A balanced BST (`std::multiset`) does all three directly. Must be a *multi*set because heights repeat.
- A max-heap (`std::priority_queue`) does insert and max but cannot delete an arbitrary element. Use **lazy deletion** (as in yesterday's sliding median): store (height, right edge), and when you look at the top, pop it if its right edge is $\le x$ (the building has already ended). Stale entries buried deeper do no harm until they surface.

**Insight 3: ties.** Several events can share the same $x$ (Example 2: one building ends at 2 and another starts at 2). If you processed them one by one and emitted a point after each, you could output $[2, 0]$ then $[2, 3]$, which is wrong. The clean fix: apply **all events at the same $x$ first**, then compare the new max with the previous max once. (A common alternative is a tie-breaking sort order: starts before ends, taller starts first, shorter ends first. It works, but the batch approach is easier to get right under interview pressure.)

**Algorithm (multiset).**

1. For each building, create events $(\text{left}, +h)$ and $(\text{right}, -h)$.

2. Sort events by $x$.
3. Keep a multiset `active` initialized with $\{0\}$ (ground), so the max is always defined.
4. For each distinct $x$: insert all $+h$, erase one copy of each $-h$. Let `cur = max(active)`. If `cur != prev`, output $(x, \text{cur})$ and set `prev = cur`.

**Complexity.** Sorting $2n$ events: $O(n \log n)$. Each multiset operation is $O(\log n)$, $2n$ of them. Total $O(n \log n)$ time, $O(n)$ space. The heap version is also $O(n \log n)$ (each building is pushed and popped at most once).

**Why the output has no duplicate heights:** we only emit when the max changes, and we emit at most one point per distinct $x$.

### 3.3 C++ solution

```cpp
// Daily Quant Digest 2026-10-19
// Algorithms & DS: Intervals and sweep line, "The Skyline Problem"
// Compile: g++ -std=c++17 -O2 -Wall 2026-10-19.cpp -o skyline && ./skyline

#include <algorithm>
#include <cassert>
#include <iostream>
#include <queue>
#include <random>
#include <set>
#include <utility>
#include <vector>

using std::vector;
using Building = std::vector<long long>;          // {left, right, height}
using Point = std::pair<long long, long long>;    // {x, height}

// ---------------------------------------------------------------------------
// Solution 1: sweep line with a multiset of active heights. O(n log n).
// ---------------------------------------------------------------------------
vector<Point> getSkyline(const vector<Building>& buildings) {
    // Each building produces two events: +h at its left edge, -h at its right edge.
    vector<Point> events;
    events.reserve(2 * buildings.size());
    for (const auto& b : buildings) {
        events.emplace_back(b[0], b[2]);    // start: positive height
        events.emplace_back(b[1], -b[2]);   // end:   negative height
    }
    // Sort by x only. All events at the same x are applied as a batch below,
    // so we do not need any clever tie-breaking rule.
    std::sort(events.begin(), events.end(),
              [](const Point& a, const Point& b) { return a.first < b.first; });

    std::multiset<long long> active{0};     // ground level is always "active"
    vector<Point> result;
    long long prevMax = 0;

    std::size_t i = 0;
    while (i < events.size()) {
        const long long x = events[i].first;
        // Apply every event that happens at this x.
        while (i < events.size() && events[i].first == x) {
            const long long h = events[i].second;
            if (h > 0) {
                active.insert(h);
            } else {
```

```
                // erase ONE copy: erase(value) would remove all equal heights
                active.erase(active.find(-h));
            }
            ++i;
        }
        const long long curMax = *active.rbegin();   // largest element
        if (curMax != prevMax) {
            result.emplace_back(x, curMax);
            prevMax = curMax;
        }
    }
    return result;
}

// ---------------------------------------------------------------------------
// Solution 2: sweep over sorted x with a max-heap and lazy deletion. O(n log n).
// Heap stores {height, right edge}; tops whose right edge <= x are stale.
// ---------------------------------------------------------------------------
vector<Point> getSkylineHeap(vector<Building> buildings) {
    std::sort(buildings.begin(), buildings.end());   // by left edge
    vector<long long> xs;
    for (const auto& b : buildings) { xs.push_back(b[0]); xs.push_back(b[1]); }
    std::sort(xs.begin(), xs.end());
    xs.erase(std::unique(xs.begin(), xs.end()), xs.end());

    std::priority_queue<Point> heap;   // max-heap on height
    vector<Point> result;
    std::size_t j = 0;
    long long prevMax = 0;
    for (long long x : xs) {
        while (j < buildings.size() && buildings[j][0] == x) {
            heap.emplace(buildings[j][2], buildings[j][1]);
            ++j;
        }
        while (!heap.empty() && heap.top().second <= x) heap.pop();   // lazy delete
        const long long curMax = heap.empty() ? 0 : heap.top().first;
        if (curMax != prevMax) {
            result.emplace_back(x, curMax);
            prevMax = curMax;
        }
    }
    return result;
}

// ---------------------------------------------------------------------------
// Brute force reference: height on each elementary interval [xs[k], xs[k+1]).
// O(n^2), used only for testing.
// ---------------------------------------------------------------------------
vector<Point> bruteForce(const vector<Building>& buildings) {
    vector<long long> xs;
    for (const auto& b : buildings) { xs.push_back(b[0]); xs.push_back(b[1]); }
    std::sort(xs.begin(), xs.end());
    xs.erase(std::unique(xs.begin(), xs.end()), xs.end());
    vector<Point> result;
    long long prev = 0;
    for (long long x : xs) {
        long long h = 0;
        for (const auto& b : buildings)
            if (b[0] <= x && x < b[1]) h = std::max(h, b[2]);
        if (h != prev) { result.emplace_back(x, h); prev = h; }
    }
    return result;
}

void print(const vector<Point>& pts) {
    std::cout << "[";
```

```cpp
    for (std::size_t k = 0; k < pts.size(); ++k)
        std::cout << (k ? "," : "") << "[" << pts[k].first << "," << pts[k].second << "]";
    std::cout << "]\n";
}

int main() {
    // Example 1
    vector<Building> ex1 = {{2, 9, 10}, {3, 7, 15}, {5, 12, 12}, {15, 20, 10}, {19, 24, 8}};
    vector<Point> want1 = {{2, 10}, {3, 15}, {7, 12}, {12, 0}, {15, 10}, {20, 8}, {24, 0}};
    auto got1 = getSkyline(ex1);
    print(got1);
    assert(got1 == want1 && getSkylineHeap(ex1) == want1);

    // Example 2: touching buildings of equal height merge into one segment
    vector<Building> ex2 = {{0, 2, 3}, {2, 5, 3}};
    vector<Point> want2 = {{0, 3}, {5, 0}};
    auto got2 = getSkyline(ex2);
    print(got2);
    assert(got2 == want2 && getSkylineHeap(ex2) == want2);

    // Example 3: identical buildings, and one hidden inside another
    vector<Building> ex3 = {{1, 5, 4}, {1, 5, 4}, {2, 3, 2}};
    vector<Point> want3 = {{1, 4}, {5, 0}};
    auto got3 = getSkyline(ex3);
    print(got3);
    assert(got3 == want3 && getSkylineHeap(ex3) == want3);

    // Example 4: a taller building starts exactly where a shorter one ends
    vector<Building> ex4 = {{1, 3, 2}, {3, 6, 5}, {6, 8, 5}};
    vector<Point> want4 = {{1, 2}, {3, 5}, {8, 0}};
    auto got4 = getSkyline(ex4);
    print(got4);
    assert(got4 == want4 && getSkylineHeap(ex4) == want4);

    // Randomised cross-check against brute force (lots of ties on purpose)
    std::mt19937 rng(20261019);
    for (int trial = 0; trial < 20000; ++trial) {
        int n = 1 + rng() % 8;
        vector<Building> bs;
        for (int k = 0; k < n; ++k) {
            long long l = rng() % 10, w = 1 + rng() % 5, h = 1 + rng() % 4;
            bs.push_back({l, l + w, h});
        }
        auto ref = bruteForce(bs);
        assert(getSkyline(bs) == ref);
        assert(getSkylineHeap(bs) == ref);
    }
    std::cout << "All tests passed.\n";
    return 0;
}
```

Output:

```
[[2,10],[3,15],[7,12],[12,0],[15,10],[20,8],[24,0]]
[[0,3],[5,0]]
[[1,4],[5,0]]
[[1,2],[3,5],[8,0]]
All tests passed.
```

### 3.4 Walkthrough of the C++

- **Type aliases with `using`.** `using Point = std::pair<long long, long long>;` is the modern replacement for `typedef`. It keeps signatures readable. `long long` avoids overflow worries since heights go up to $2^{31} - 1$ and we negate them (and sums or differences of coordinates near $2^{31}$ would overflow an `int`).
- **`reserve` and `emplace_back`.** `events.reserve(2 * n)` allocates once, so the $2n$ insertions never reallocate. `emplace_back(b[0], b[2])` constructs the `pair` in place from its arguments instead of building a temporary and copying it.
- **`const auto& b : buildings`.** A range-based for loop that binds each element by const reference. Writing `auto b` would copy each inner `vector` (a heap allocation per building). This is one of the most common silent performance bugs in interview C++.
- **Lambda comparator.** `std::sort(..., [](const Point& a, const Point& b) { return a.first < b.first; })` sorts by $x$ only. The `[]` is an empty capture list (the lambda uses nothing from the enclosing scope). Because we batch by $x$ afterwards, the order within the same $x$ is irrelevant, so we do not even need `stable_sort`. The comparator must be a strict weak ordering: use `<`, never `<=`, or `std::sort` has undefined behavior.
- **`std::multiset<long long> active{0};`** A red-black tree that keeps elements sorted and allows duplicates. Brace-initialization inserts the ground level 0.
- **`active.erase(active.find(-h))`, the classic trap.** `multiset::erase(value)` removes **every** element equal to `value` (and returns how many). If two buildings of height 15 overlap and one ends, `erase(15)` would delete both. `find` returns an iterator to one copy, and `erase(iterator)` removes exactly that one. Each building's height was inserted before its end event, so `find` cannot return `end()` here.
- **`*active.rbegin()`.** `rbegin()` is a reverse iterator to the last (largest) element; dereferencing it gives the max in $O(1)$. `*std::prev(active.end())` is equivalent.
- **Two-level `while` loop.** The outer loop picks the next $x$; the inner loop consumes every event with that $x$. Using an explicit index `i` shared by both loops is the idiomatic way to do "group by" over a sorted vector.
- **`std::priority_queue<Point>`.** A max-heap. `std::pair` compares lexicographically, so the top is the tallest building (ties broken by larger right edge, which is harmless). Lazy deletion: `while (!heap.empty() && heap.top().second <= x) heap.pop();` throws away buildings that have already ended. Always check `empty()` before `top()`: calling `top()` on an empty heap is undefined behavior, not an exception.
- **Sort-unique-erase idiom.** `std::unique` moves consecutive duplicates to the end and returns an iterator to the new logical end; `erase` then shrinks the vector. It only removes *adjacent* duplicates, so sort first.
- **`std::sort` on `vector<vector<long long>>`.** Vectors compare lexicographically, so `buildings` gets sorted by left edge (then right, then height).
- **Testing with `assert` and `std::mt19937`.** The random test generates small buildings with many ties (coordinates in $[0, 14]$, heights in $[1, 4]$) and checks both solutions against the $O(n^2)$ brute force 20,000 times. Small ranges are deliberate: they maximize collisions, which is where sweep-line bugs live. `vector<pair>` supports `==`, so comparing whole outputs is one line.

### 3.5 Variations to be ready for

- **Meeting Rooms II** (minimum number of rooms = maximum overlap): same events, but the "data structure" is just a counter. Sort events with ends before starts at equal times for half-open intervals.
- **Merge Intervals / Insert Interval:** sort by start, sweep, extend the current block while the next start $\leq$ current end.
- **Employee Free Time:** merge everyone's intervals, then output the gaps.
- **Area of the union of rectangles:** sweep in $x$, and maintain the covered length in $y$ with a segment tree over compressed $y$ coordinates (curriculum topic 16). The skyline is the special case where all rectangles touch the ground, so a max replaces the coverage count.
- **Order book analogy:** the multiset of active heights is like a price level book where you need the best bid under inserts and cancels. Real matching engines use the same trick (a sorted structure keyed by price, or a heap with lazy cancellation).

---

## 4. Music: Pink Floyd, "Wish You Were Here" (1975)

**The song.** The title track of Pink Floyd's 1975 album *Wish You Were Here*, the follow-up to *The Dark Side of the Moon*. Music by David Gilmour and Roger Waters, lyrics by Waters. About five and a half minutes long.

**The story behind it.** Pink Floyd was started in the 1960s around Syd Barrett, a brilliant singer and songwriter. By 1968 his mental health had declined so much that the band had to carry on without him. Seven years later, the band was rich and famous, but Waters felt they had lost something: friendship, honesty, the joy of just playing. The album is about **absence**: Syd's absence, and the band members' own feeling of being absent from their own lives while the music business turned them into a product. The famous cover, two businessmen shaking hands while one of them is on fire, is about that same emptiness ("getting burned" in a deal).

Then something eerie happened. During the recording sessions at Abbey Road in June 1975, a heavy man with a shaved head and shaved eyebrows wandered into the studio. Nobody recognised him for a while. It was Syd Barrett. Several band members were reportedly in tears. He left, and most of them never saw him again.

**Why people love it.**

- **It starts like a memory.** The song opens with the sound of someone tuning a car radio, and a 12-string guitar playing thin and far away, as if coming out of that radio. Then a second guitar comes in, close and warm, playing along, like a person in the room joining in with a song from the past. In ten seconds, without a word, the song tells you its theme: someone present, playing along with someone who is not.

- **It is simple on purpose.** A handful of basic chords, a gentle tempo, an acoustic guitar. Pink Floyd were known for long, complex, spacey pieces; this is the opposite. It is the band at its most human, which is why it is one of the first songs guitar learners play around a campfire.
- **The lyrics are questions, not answers.** The verses ask whether you can really tell the difference between things that look alike, the real thing and a good imitation of it. It works for Syd, for a lost friend, for a breakup, or for anyone who sold a bit of themselves for a “good deal”.

**How to listen to it.** Use headphones. Notice the moment the second guitar enters and the sound becomes full: that is the switch from “radio” to “real”. Near the end, listen for Gilmour singing the guitar melody without words, alongside his own guitar, and for the wind sound that fades the song out (on the album, it leads back into “Shine On You Crazy Diamond”, the other song written for Syd).

**If you like it, next try:** “Shine On You Crazy Diamond (Parts I to V)”, same album, for the full tribute.